

MongoDB

Complete Interview Revision Guide

Prepared for **Satish Kumar**

SDE-3 Node.js — WeKan Enterprise Solutions

August 2026

WeKan is a strategic partner of MongoDB, doing enterprise data migrations and legacy modernisation. MongoDB depth is the differentiator in this interview.

Contents

1. **Indexes** — what they are, sorts, direction, ESR
2. **explain()** — reading the output
3. **Index rules** — leftmost prefix, multikey, write cost
4. **\$indexStats** — finding dead indexes
5. **RAM vs disk** — the WiredTiger cache
6. **Data modeling** — embed vs reference
7. **Schema patterns** — extended reference, computed, bucket, attribute
8. **Replication** — replica sets, oplog, elections, write concern
9. **Transactions** — when you need one, and when you do not
0. **Concurrency** — atomic updates, lost updates, optimistic locking
1. **Change streams** — resume tokens and the pod problem
2. **MongoDB in practice** — what the industry actually does
3. Appendix — one-liners and self test

Parked for later: aggregation pipelines — `$group` , `$project` , `$lookup` , `$unwind` , materialized views via `$out` / `$merge` .

Box colours — **yellow** are your own questions answered in place, **blue** are the sentences to say aloud, **red** are traps.

1. Indexes

YOUR QUESTION — WHAT DID "REGISTER" MEAN?

That was my word, not a MongoDB term. I meant a **register book** — a shop ledger, the thick notebook where a shopkeeper writes entries in order.

That is all an index is: **a notebook kept in sorted order, separate from the actual data.**

Your `orders` collection is the pile of order slips, in whatever order they arrived. The index on `amount` is a notebook where every amount is already written in sorted order, each with a pointer to the real slip:

```
Rs 48,900 -> slip #9921
Rs 47,200 -> slip #1043
Rs 46,850 -> slip #7788
```

MongoDB maintains this notebook automatically on every insert and update. "Read the register" means "walk the index."

A sort without a supporting index

```
db.orders.find({}).sort({ amount: -1 }).limit(10)
```

Without an index, MongoDB must read all 5 lakh documents, load them into memory, sort all 5 lakh, throw away 4,99,990, and return 10. This is a **blocking in-memory sort** — blocking because it cannot return the first result until it has seen the last document.

And it can simply fail. MongoDB caps in-memory sorts at **32 MB**:

```
Sort exceeded memory limit of 33554432 bytes
```

Your endpoint returns a 500. A real production error, not a theoretical one.

With an index, the same query means: open the notebook, read the first 10 lines, stop. No scanning, no sorting, no 32 MB risk.

```
db.orders.createIndex({ amount: -1 })
```

Index direction

YOUR QUESTION — WHAT IF THE SORT IS ASCENDING?

Same index still works. **A single-field index can be read in both directions.** To get the cheapest orders, MongoDB reads the notebook from the bottom upward.

```
Rs 48,900 -> order_9921      ^
Rs 47,200 -> order_1043     ^  read upward for ascending
...                         ^
Rs   350 -> order_2210      <- start here for ascending
Rs   299 -> order_0087      <-
```

So `createIndex({ amount: -1 })` supports **both** `sort({ amount: -1 })` and `sort({ amount: 1 })`. You do not need two indexes.

Where direction does matter: compound indexes. Given `{ city: 1, amount: -1 }` the notebook is grouped by city and, within each city, sorted high to low:

```
Jamshedpur, Rs 48,900
Jamshedpur, Rs 12,300
Jamshedpur, Rs   350
Ranchi,     Rs 47,200
Ranchi,     Rs  8,100
```

```
.sort({ city: 1, amount: -1 })  works — read straight down
.sort({ city: -1, amount: 1 }) works — read straight up, all flipped
.sort({ city: 1, amount: 1 })  FAILS — in-memory sort
```

The rule: you can read an index forwards or backwards, but you must **flip every field, or none**. Mixed directions need their own index.

The ESR rule

The single most-asked MongoDB index question.

```
db.orders.find({ city: "Jamshedpur", amount: { $gt: 5000 } })
      .sort({ createdAt: -1 })
```

Three fields. Which order in the index? **E, then S, then R:**

```
db.orders.createIndex({ city: 1, createdAt: -1, amount: 1 })
```

Letter	Means	Why it goes there
E	Equality	All Jamshedpur rows sit in one continuous block. MongoDB jumps straight to it and ignores the rest of the notebook. Huge cut.
S	Sort	Inside that block, rows are already newest-first. MongoDB reads them in order — no sort stage at all . This is the key win.
R	Range	Now it simply skips rows where amount is too low as it walks.

What breaks if range goes in the middle

```
db.orders.createIndex({ city: 1, amount: 1, createdAt: -1 }) // E, R, S – wrong
```

```
Jamshedpur, Rs 5,100, 2026-08-02
Jamshedpur, Rs 8,900, 2026-08-14
Jamshedpur, Rs 12,300, 2026-08-16
```

A range match gives you **many** amount values. Within each one dates are sorted, but *across* them the dates are jumbled. So MongoDB must collect all matches and sort them in memory. You are back to a blocking sort, and `explain()` shows a `SORT` stage.

THE REASON IN ONE LINE

A range spreads your results across many index values, so any sort field placed after it is no longer in order.

SAY THIS

"ESR — equality, sort, range. Equality fields go first because they narrow to one contiguous block of the index. The sort field goes next so the index delivers documents already ordered and MongoDB skips the sort stage. Range fields go last, because a range spans many index values and destroys ordering for anything after it. I would confirm with `explain()` — I want `IXSCAN` and no `SORT` stage."

Worked example. Query: `find({ status: "delivered", createdAt: { $gte: d } }).sort({ amount: -1 })`
Index: `{ status: 1, amount: -1, createdAt: 1 }`

Note the amount direction is -1, matching the sort. The `createdAt` direction does not matter — it is a range filter, not a sort, so MongoDB skips non-matching rows either way.

2. explain()

YOUR QUESTION — WHAT DO I ACTUALLY GET FROM EXPLAIN()?

Add `.explain("executionStats")` to any query. That is the mode you want — the default only shows the plan, not what happened. The output is a large JSON blob; **ignore 95% of it**. Four fields matter.

```
db.orders.find({ city: "Jamshedpur" })
  .sort({ amount: -1 })
  .explain("executionStats")
```

Field	Tells you	Good value
stage	How it found the data	IXSCAN
totalDocsExamined	Documents read	close to nReturned
nReturned	Documents returned	—
executionTimeMillis	Time taken	low

Bad result

```
{
  "executionStats": {
    "nReturned": 10,
    "totalDocsExamined": 500000,
    "executionTimeMillis": 812,
    "executionStages": {
      "stage": "SORT",
      "inputStage": { "stage": "COLLSCAN" }
    }
  }
}
```

Examined 5 lakh to return 10. `COLLSCAN` means it read the whole pile. `SORT` means it sorted in memory. **No index.**

Good result

```
{
  "executionStats": {
    "nReturned": 10,
    "totalDocsExamined": 10,
    "executionTimeMillis": 2,
    "executionStages": {
      "stage": "FETCH",
      "inputStage": { "stage": "IXSCAN" }
    }
  }
}
```

Examined 10 to return 10. `IXSCAN` means it walked the notebook. **No SORT stage at all** — that is the win.

Stage	Meaning
COLLSCAN	Read every document. Bad on large collections.
IXSCAN	Walked the index. Good.
SORT	Sorted in memory. Your index does not support the sort.
FETCH	Went from index entry to the real document. Normal.

Bonus: `IXSCAN` with **no** `FETCH` is a **covered query** — the index alone had every field needed, so MongoDB never touched the documents. Fastest possible. Worth naming in an interview.

SAY THIS — THE WHOLE SKILL IN ONE SENTENCE

"I would run `explain('executionStats')` and check that `totalDocsExamined` is close to `nReturned`, and that there is no `SORT` stage. A large gap between examined and returned means the index is not being used properly."

3. Index rules

Leftmost prefix

An index on `{ city: 1, amount: -1, createdAt: 1 }` also works as an index on:

- `{ city }` — yes
- `{ city, amount }` — yes
- `{ amount }` alone — **no**
- `{ createdAt }` alone — **no**

Notebook logic: entries are grouped by city first. If you do not know the city, you cannot jump to a block — you would have to read the whole notebook.

PRACTICAL VALUE

One well-ordered compound index often replaces three separate indexes. This is the answer to "you have 8 indexes on this collection, how would you reduce them?"

Multikey indexes (arrays)

```
{ _id: 1, tags: ["kurta", "cotton", "festive"] }
```

An index on `{ tags: 1 }` creates **one index entry per array element** — three entries pointing to the same document. That is a **multikey index**.

- You **cannot** have a compound index on two array fields — the combinations explode
- Multikey indexes **cannot fully cover** a query

Indexes cost writes

Every index must be updated on every insert and update. Eight indexes means eight extra writes per insert.

SAY THIS

"Indexes speed up reads and slow down writes. On a write-heavy collection I would audit unused indexes with `$indexStats` before adding more."

4. \$indexStats

```
db.orders.aggregate([{$indexStats: {}}])
```

An aggregation stage returning **how many times each index has actually been used** since the server last restarted.

```
[
  {
    name: "id_",
    accesses: { ops: 84200, since: "2026-06-01T00:00:00Z" } },
  {
    name: "city_1_amount_-1",
    accesses: { ops: 12750, since: "2026-06-01T00:00:00Z" } },
  {
    name: "customerEmail_1",
    accesses: { ops: 0, since: "2026-06-01T00:00:00Z" } }  // never used
]
```

`customerEmail_1` has **0 ops in two and a half months**. Somebody added it for a feature that got removed. It is still slowing down every insert and update, and consuming RAM and disk.

```
db.orders.dropIndex("customerEmail_1")
```

THE TWO TRAPS — THE SECOND IS THE ONE PEOPLE MISS

1. Counters reset on restart. `ops` counts since the last `mongod` start, not forever. If the server restarted yesterday, everything looks unused. **Always check** `since` before concluding an index is dead. Run this on a server with weeks of uptime.

2. Zero does not always mean useless. A monthly reporting query or a quarterly reconciliation job may legitimately show 0 ops today. **Check with the team before dropping.**

Also: in a replica set, run it on each node separately — the primary and secondaries serve different traffic.

SAY THIS

"I would run `$indexStats` to find indexes with zero or near-zero ops, checking the `since` field so I am not misreading a recent restart. Unused indexes cost write throughput and RAM without returning anything. Alongside that I would look for indexes made redundant by the leftmost prefix rule — if I have `{city, amount}` I do not also need `{city}`."

5. RAM vs disk — the WiredTiger cache

YOUR QUESTION — ARE INDEXES ON RAM OR HARD DISK?

Both — and that is the whole point.

Indexes live **on disk** permanently. That is the source of truth and it survives restarts. But MongoDB cannot search a disk file directly — to use an index it must **load those pages into RAM**.

MongoDB reserves a chunk of RAM called the **WiredTiger cache**, by default **50% of (RAM minus 1 GB)**. On a 16 GB machine, roughly 7.5 GB. **Documents and indexes compete for space in that same cache.**

Warm — index page already in cache — served from RAM, microseconds.

Cold — read from disk, evict something else to make room, then use it — milliseconds. **100 to 1000 times slower.**

Where a dead index actually hurts

Your cache is 7.5 GB. Your working indexes need 6 GB. Fine — everything stays warm.

Now add the unused `customerEmail_1` index, 2 GB. Total 8 GB, cache is 7.5 GB.

MongoDB now **evicts pages constantly**. A live index page gets thrown out to make room, then a query needs it back, so something else gets thrown out. This is **cache thrashing**, and p99 latency jumps.

THE POINT

The dead index is not just idle overhead. **It is actively pushing your working indexes out of RAM.**

```
db.orders.totalIndexSize()    // bytes, all indexes on this collection
db.orders.stats().indexSizes  // per-index breakdown
```

Compare against your cache size. If total index size exceeds it, you have a problem before you have even counted documents.

SAY THIS — THE PHRASE THEY LISTEN FOR

"Indexes are stored on disk but must be paged into the WiredTiger cache to be used. The rule of thumb is that **the working set — hot indexes plus hot documents — should fit in that cache**. Once indexes spill to disk you get cache eviction and latency falls off a cliff, which is why unused indexes are a real cost and not just clutter."

6. Data modeling — embed vs reference

This is the topic most likely to decide the interview. WeKan is a MongoDB partner doing enterprise migrations — data modeling is what they get paid for. The first question is always: embed or reference?

The two choices

Embed — put it inside:

```
{
  _id: "order_1",
  customer: "Priya",
  items: [
    { name: "Cotton Kurta", qty: 1, price: 2499 },
    { name: "Silk Dupatta", qty: 2, price: 899 }
  ]
}
```

Reference — point to it:

```
// orders collection
{ _id: "order_1", customer: "Priya", itemIds: ["item_1", "item_2"] }

// items collection
{ _id: "item_1", name: "Cotton Kurta", qty: 1, price: 2499 }
{ _id: "item_2", name: "Silk Dupatta", qty: 2, price: 899 }
```

Why embed is usually right

This is the mindset shift from SQL. In MySQL you would **always** split these into two tables — that is normalization, and it is drilled into you.

MongoDB says: do not.

Reading the embedded order is **one query, one disk read**. Everything you need arrives together. Reading the referenced version is **two queries** — fetch the order, then fetch the items. Or a `$lookup`, which is MongoDB's join and is slower than a real SQL join.

THE RULE

Data that is read together should be stored together.

When embedding breaks — three situations

1. The array grows without limit. A document has a **16 MB hard cap**. Embed 50,000 items and the write fails. Worse, MongoDB reads the *whole* document — a 2 MB order is 2 MB off disk even if you only wanted the customer name. This is the **unbounded array anti-pattern**.

2. The data is shared. Product details embedded in every order means the kurta's description is copied into 10,000 order documents. Change it and you update 10,000 places.

3. You need to query the child on its own. "Show me all Cotton Kurtas sold this month, across all orders." Awkward if embedded; trivial as its own collection.

The decision, in order

1. **Does the child ever get accessed alone?** Yes → reference.
2. **Can the array grow unbounded?** Yes → reference.
3. **Otherwise** → embed.

What the interviewer is actually listening for

You lead with the access pattern, not the entities. The wrong instinct is to draw entities and relationships like an ER diagram. The right one is: "*How does the application read this? What is the most frequent query?*" In MongoDB you design the schema **backwards from the queries**, not forwards from the data.

You accept duplication. SQL people flinch at storing the same value twice. In MongoDB, duplicating a product name into an order is often correct — and there is a second reason beyond speed: an order should record the price **at the time of purchase**. If you reference the product and the price changes, your old orders silently change too. **Duplication is the correct answer there, not a compromise.**

Worked example — product reviews

A popular kurta gets 5,000 reviews. Embed or reference?

Reference — unbounded growth, 16 MB cap, and reviews are queried independently ("all reviews by this user"). But the strong answer is that **you do not have to choose one or the other**:

```
// products collection
{
  _id: "prod_1",
  name: "Cotton Kurta",
  price: 2499,
  reviewCount: 5127,           // denormalized
  avgRating: 4.3,             // denormalized
  recentReviews: [            // last 3 only, embedded
    { user: "Priya", rating: 5, text: "Lovely fabric" },
    { user: "Meena", rating: 4, text: "True to size" }
  ]
}

// reviews collection – the full set
{ _id: "rev_1", productId: "prod_1", user: "Priya", rating: 5, text: "..." }
```

Why this wins: the product page needs name, price, rating and a few sample reviews — now **one query**. "See all 5,127 reviews" triggers a second, paginated query only when the user asks. And `reviewCount` / `avgRating` are stored rather than computed, so you never aggregate over 5,127 documents to render a page.

YOUR QUESTION — HOW DOES THE ARRAY STAY BOUNDED AT 3?

Unbounded — the problem: at ~3 KB per review, 5,127 reviews is ~15 MB. One more review and the write fails at the 16 MB cap. And every product page load reads all 15 MB off disk just to show a name and price.

Bounded — the fix: the document stays around 10 KB **forever**, whether there are 5,000 reviews or 5 lakh.

MongoDB does this in one operation:

```
db.products.updateOne(
  { _id: "prod_1" },
  {
    $push: {
      recentReviews: {
        $each: [ newReview ],
        $slice: -3           // keep only the last 3
      }
    },
    $inc: { reviewCount: 1 }
  }
);
```

`$slice: -3` pushes the new review and **drops the oldest**, so the array never exceeds 3. Automatic, no cleanup job.

The phrase to use: "I would cap the embedded array with `$slice` so the document size stays constant regardless of how much the child data grows."

State the trade-off out loud. Denormalized data can go stale — `avgRating` is only correct if you update it on every new review. That is a write-time cost you are accepting to make reads fast.

"I am trading write complexity for read performance, which is the right trade when reads outnumber writes by 1000 to 1, as they do on a product page."

Anyone can denormalize; a senior explains what it costs.

SAY THIS

"MongoDB schema design is driven by access patterns, not normalization. The default is to embed, because data read together should be stored together and that makes it a single-document read. I would reference when the child is queried independently, when the array can grow unbounded toward the 16 MB document limit, or when the data is shared and updated in one place."

7. Schema design patterns

Four named patterns. **Naming them is worth a lot** — it signals you have read MongoDB's own material, which matters at a MongoDB partner.

1. Extended Reference

Problem: you referenced, so now you need a second query to display anything useful.

```
{ _id: "order_1", customerId: "cust_9" }  
// must query customers just to show "Priya" on the order list
```

Fix — copy the few fields you always display:

```
{  
  _id: "order_1",  
  customerId: "cust_9",  
  customer: { name: "Priya", city: "Jamshedpur" }  // duplicated  
}
```

One query renders the order list. **Copy only fields that rarely change** — a name, not a phone number.

2. Computed / Precomputed

Problem: you are aggregating on every page load.

```
{ _id: "prod_1", reviewCount: 5127, avgRating: 4.3 }
```

Update these on write instead of computing them on read.

THE PRINCIPLE BEHIND BOTH

Reads happen 1000 times more often than writes, so pay the cost at write time.

3. Bucket — for time-series data

Very relevant to WeKan — enterprise IoT and event data is a common migration workload.

Problem: one document per event. 1 crore sensor readings means 1 crore documents. The index is huge, queries are slow, and per-document overhead dominates.

```
// bad — one doc per reading  
{ sensorId: "s1", ts: "2026-08-16T10:00:00Z", temp: 31.2 }  
{ sensorId: "s1", ts: "2026-08-16T10:01:00Z", temp: 31.4 }
```

Fix — one document per hour, readings inside:

```
{  
  sensorId: "s1",  
  hour: "2026-08-16T10:00:00Z",  
  count: 60,  
  readings: [  
    { min: 0, temp: 31.2 },  
    { min: 1, temp: 31.4 }  
  ]  
}
```

1 crore documents becomes about 1.7 lakh. **Bounded at 60** — the hour boundary caps it naturally. The index is tiny, and reading an hour is one document instead of 60.

4. Attribute

Problem: products have wildly different fields and you cannot index them all.

```
{ name: "Kurta", fabric: "cotton", sleeve: "full", neckline: "round" }  
{ name: "Saree", fabric: "silk", borderWidth: "3in", weave: "banarasi" }
```

Fix — turn fields into an array of key-value pairs:

```
{  
  name: "Kurta",  
  attrs: [  
    { k: "fabric", v: "cotton" },  
    { k: "sleeve", v: "full" },  
    { k: "neckline", v: "round" }  
  ]  
}  
  
db.products.createIndex({ "attrs.k": 1, "attrs.v": 1 })  
db.products.find({ attrs: { $elemMatch: { k: "fabric", v: "silk" } } })
```

One index instead of twenty, and new attributes need no schema change.

Pattern	Solves
Extended Reference	Avoid a second query after referencing
Computed	Avoid aggregating on every read
Bucket	Too many small documents (time-series, logs)
Attribute	Too many varied fields to index

SAY THIS

"MongoDB has a catalogue of schema patterns. The ones I reach for most are extended reference and computed fields to keep reads to a single query, bucketing for time-series data to avoid millions of tiny documents, and the attribute pattern when documents have highly variable fields that would otherwise need one index each."

8. Replication

A **replica set** is a group of MongoDB servers holding the **same data**. Standard setup is **3 nodes**: one primary, two secondaries.

```
Primary    <- all writes go here
|
|
Secondary  <- copy
Secondary  <- copy
```

- **All writes go to the primary.** Always. Only one primary exists.
- **Secondaries copy** from the primary by replaying its operation log, the **oplog**.
- **Reads default to the primary**, but can be redirected.

The oplog

A **capped collection** on the primary called `oplog.rs` that records every write operation.

```
{ op: "i", ns: "shop.orders", o: { _id: "order_1", amount: 2499 } }
{ op: "u", ns: "shop.orders", o2: { _id: "order_1" },
  o: { status: "shipped" } }
```

Secondaries **tail** it — read new entries continuously and replay them locally. **That is all replication is.**

It is idempotent. `$inc: { count: 1 }` gets rewritten into `$set: { count: 6 }` before entering the oplog, so replaying the same entry twice is safe.

It is capped — fixed size. If a secondary is offline too long, the oplog wraps around and overwrites entries it still needed. That secondary is now **"too stale"** and must be fully resynced from scratch. Oplog sizing is a real operational concern.

Why three nodes and not two

Automatic failover. The primary dies, the remaining nodes hold an **election** and vote a new primary in within about 10 seconds. Your app reconnects automatically.

YOUR QUESTION — WHAT HAPPENS WITH ONLY ONE REPLICA AND THE PRIMARY DIES?

Writes stop. The database goes read-only.

The surviving secondary needs a **majority** to elect itself. Majority of 2 is 2. It has 1 vote. It cannot win, so it stays a secondary.

```
Before: Primary + Secondary    (2 nodes, majority = 2)
After:  Secondary alone        (1 vote out of 2 – not a majority)
Result: No primary. No writes. Reads only.
```

Your data is safe. Your application is down for writes. Someone must manually intervene.

Why the majority rule exists: to prevent **split-brain**. Without it, a network partition would leave both nodes thinking they are primary — two servers accepting conflicting writes with no way to reconcile. Requiring a majority guarantees only one side can ever win, because both sides cannot simultaneously hold more than half.

The cheap fix — an arbiter. An arbiter holds no data and only votes. Majority becomes 2 of 3, so the secondary plus arbiter can elect. **But say the caveat:** with an arbiter you only have **two** data copies, so losing one makes `w: "majority"` writes start failing. A real third node is always better.

Read preference

```
db.orders.find({}).readPref("secondaryPreferred")
```

Setting	Behaviour
primary	Default. Always consistent.
secondaryPreferred	Secondaries if available, else primary
nearest	Lowest latency

Secondaries lag. Replication is asynchronous — a secondary may be milliseconds or seconds behind.

The classic bug: Priya places an order (written to primary), the page redirects, the read goes to a secondary that has not caught up, and **her order is not there**. She places it again.

Rule: use secondary reads for reporting and analytics. **Never for read-after-write in the same user flow.**

Write concern — how safe is a write

```
db.orders.insertOne(order, { writeConcern: { w: "majority" } })
```

Setting	Means
w: 1	Primary acknowledged. Fast. Lost if the primary dies before replicating.
w: "majority"	2 of 3 nodes have it. Survives a primary failure.
j: true	Written to the journal on disk, not just memory

Use `majority` for orders and payments; `w: 1` is fine for analytics events. **This is the same acks and replication trade-off as Kafka** — say that, it shows the concept transfers.

Replication vs sharding — they will check this

	Replication	Sharding
Solves	Availability and durability	Data too big for one machine
Each node has	The same data	A different slice
Scales	Reads	Reads and writes

REMEMBER IT AS
Replication is **copies**. Sharding is **splits**. Candidates mix these up constantly.

What people actually run

Standard production: 3 data-bearing nodes — one per availability zone. Survives losing an entire AZ. This is the default on MongoDB Atlas and what most companies run.

Variation	Why
5 nodes	Critical systems. Survives losing two nodes. Also lets you dedicate secondaries to analytics.
3 + hidden secondary	Hidden node takes no reads and cannot become primary. For backups or heavy reporting without touching production.
3 + delayed secondary	Deliberately kept an hour behind. If someone runs a bad deleteMany, you have a copy from before the mistake.
Multi-region	2 in Mumbai, 1 in Singapore. Disaster recovery, at the cost of election latency.

What is rare: arbiters (officially discouraged), 2 nodes (never in production), and **even numbers** — 4 nodes has the same fault tolerance as 3, since majority is 3 either way, so you pay for a server that buys you nothing.

SAY THIS
"A replica set is typically three nodes across three availability zones — one primary taking all writes, two secondaries replaying the oplog. An odd number is needed so a majority can elect a new primary on failover. I would use `w:'majority'` for anything transactional, and only route reads to secondaries for reporting, because replication lag breaks read-after-write consistency."

9. Transactions

The problem

```
await db.orders.insertOne({ customer: "Priya", amount: 2499 });
await db.inventory.updateOne({ _id: "kurta" }, { $inc: { stock: -1 } });
```

The server crashes between them. **The order exists but stock was never reduced.** Your inventory is now wrong, silently, forever.

The MongoDB answer that comes first

SAY THIS FIRST

"My first move is schema design. **A single-document write is already atomic**, so most cases that would need a transaction in SQL do not need one in MongoDB."

That is the answer showing you think in MongoDB rather than translating from SQL.

When you genuinely need one

Two collections must change together and cannot be merged into one document. Orders and inventory is the real example — inventory is shared across all orders, so it cannot be embedded.

```
const session = client.startSession();

try {
  await session.withTransaction(async () => {
    await db.orders.insertOne({ customer: "Priya", amount: 2499 }, { session });
    await db.inventory.updateOne(
      { _id: "kurta" }, { $inc: { stock: -1 } }, { session }
    );
  });
} finally {
  await session.endSession();
}
```

Every operation must pass { session } — miss it on one and that write silently escapes the transaction. Common bug.

The three constraints to name

1. **Requires a replica set.** Transactions do not work on a standalone MongoDB. Fine in production, catches people on local dev.
2. **60-second default limit.** Exceed it and the transaction aborts. Transactions are for a handful of documents, not bulk operations.
3. **They cost performance.** MongoDB holds locks and snapshots for the duration. Under contention throughput drops noticeably. This is why "just use transactions everywhere" is the wrong instinct.

Version note: single-replica-set transactions landed in 4.0, cross-shard in 4.2.

What the industry does

Transactions are used sparingly. Most Node and Mongo codebases have a handful — payments, inventory, financial ledgers — and nothing else. Two alternatives are far more common:

- **Design it away.** Restructure so the atomic unit is one document. The most common fix by a wide margin.
- **Eventual consistency with compensation.** Write the order, publish an event to Kafka or RabbitMQ, let a consumer update inventory. If it fails, retry or run a compensating action. This is the **Saga pattern**, and it is what most microservice systems do — because a transaction cannot span two services anyway.

SAY THIS

"MongoDB gives single-document atomicity for free, so my first move is schema design — most SQL-style transaction needs disappear if the atomic unit is one document. When two collections genuinely must change together I would use a session-based transaction, keeping it short and passing the session to every operation. In a distributed system I would usually prefer eventual consistency with a compensating action, since a transaction cannot span services."

10. Concurrency

YOUR QUESTION — IF MANY REQUESTS UPDATE THE SAME RECORD, DO ALL BUT ONE FAIL?

No — nobody fails. They queue.

Atomicity means **no half-finished update is ever visible**. It does not mean only one writer wins. MongoDB serialises them:

```
Request A -> applies -> done
Request B -> applies -> done
Request C -> applies -> done
```

All three succeed, one after another. Three `$inc: { stock: -1 }` on stock 10 gives **7**, because `$inc` is applied by the server so nothing is lost.

Inserts are atomic too. And here duplicates genuinely do fail — but only because of a unique index, not concurrency:

```
db.users.createIndex({ email: 1 }, { unique: true })
// second insert with same email -> E11000 duplicate key error
```

Without a unique index, both inserts succeed and you have two documents.

Where you DO lose data

```
const doc = await db.inventory.findOne({ _id: "kurta" }); // both read 10
doc.stock = doc.stock - 1;
await db.inventory.replaceOne({ _id: "kurta" }, doc);      // both write 9
```

Two requests, stock should be 8, you get **9**. This is a **lost update** — and MongoDB's atomicity did not save you, because the read and the write were two separate operations.

The fix — push the logic into the query

Level 1 — atomic operators. Let the server compute:

```
db.inventory.updateOne({ _id: "kurta" }, { $inc: { stock: -1 } });
```

But this can go negative. Stock 0 becomes -1. So:

Level 2 — put the condition in the filter:

```
const result = await db.inventory.updateOne(
  { _id: "kurta", stock: { $gte: 1 } }, // the check lives here
  { $inc: { stock: -1 } }
);

if (result.modifiedCount === 0) {
  throw new Error("Out of stock");
}
```

THIS IS THE KEY MOVE

The filter and the update are **one atomic operation** — there is no gap for another request to slip into. If stock is 0 the filter does not match, nothing is modified, and `modifiedCount` is 0. Two requests, one item left: one succeeds, one gets 0. Correct, with no lock and no transaction.

Level 3 — when you need the document back:

```
const doc = await db.inventory.findOneAndUpdate(
  { _id: "kurta", stock: { $gte: 1 } },
  { $inc: { stock: -1 } },
  { returnDocument: "after" }
);

if (!doc) throw new Error("Out of stock");
```

Same atomicity, and you get the updated document in the same round trip. **This is the workhorse** — most real code uses `findOneAndUpdate` for exactly this.

Optimistic locking — when one query is not enough

```
const doc = await db.orders.findOne({ _id: id });
const newTotal = complexCalculation(doc);

const result = await db.orders.updateOne(
  { _id: id, version: doc.version },           // only if unchanged
  { $set: { total: newTotal }, $inc: { version: 1 } }
);

if (result.modifiedCount === 0) {
  // someone else changed it – re-read and retry
}
```

If another request modified the document in between, the version no longer matches, the update does nothing, and you retry. **Same pattern as JPA's `@Version` in Spring.**

YOUR QUESTION — IS `__v` THE SAME THING?

No. Two precisions:

1. `__v` is Mongoose's, not MongoDB's. The native driver knows nothing about it. Mongoose adds it to every document automatically.

2. It only guards array positional updates. If you `save()` a document where an array element was modified by index, Mongoose includes `__v` in the query so that if someone else shifted the array in between, the save fails rather than writing to the wrong position.

That is its entire scope — it does not protect ordinary field updates.

So **yes**, for real optimistic concurrency you add your own field. Name it `version` or `rev` — anything but `__v`, to avoid colliding with Mongoose's own handling.

What the industry does: `findOneAndUpdate` with the condition in the filter is the default and covers most cases. Optimistic locking with a version field for complex updates. **Pessimistic locking is rare** in MongoDB — if you truly need a lock, teams reach for **Redis** (`SET key NX PX`, or Redlock) rather than trying to lock in MongoDB.

SAY THIS

"I would move the condition into the query filter so the check and the update are one atomic operation — `findOneAndUpdate` with the guard in the filter, treating a zero `modifiedCount` as the conflict signal. Where the new value needs computing in application code, I would use a version field for optimistic concurrency and retry on mismatch."

11. Change streams

The problem: Priya places an order. Three things must happen after — confirmation email, analytics update, warehouse notification. Doing it all in the request handler makes the order API slow, and if the email service is down the order fails. Polling every 30 seconds is wasteful and always up to 30 seconds late.

Change streams push changes to you

```
const stream = db.orders.watch();

stream.on("change", (change) => {
  if (change.operationType === "insert") {
    sendConfirmationEmail(change.fullDocument);
  }
});
```

Insert an order anywhere — your app, a script, the Atlas UI — and this fires within milliseconds.

How it works: change streams tail the **oplog**, the same log secondaries replay for replication. So it **requires a replica set**, same as transactions.

Filter at the database

```
db.orders.watch([
  { $match: { operationType: "insert",
    "fullDocument.amount": { $gte: 10000 } } }
]);
```

Only high-value orders reach your app. Filtering server-side saves bandwidth and CPU.

Resume tokens — the part that matters

Your service restarts. Changes happened while it was down. **They are lost** unless you handle this.

Every change carries an `_id` called a **resume token**. Store it, and restart from there:

```
const stream = db.orders.watch([], { resumeAfter: savedToken });
```

You must persist the token — in Redis or a MongoDB collection. This is the single most-asked follow-up on change streams, and it is **the same idea as a Kafka consumer offset**.

The limit: resume tokens only work while the entry still exists in the oplog. Down longer than your oplog window and you cannot resume — you need a full reconciliation instead.

Change streams vs Kafka

	Change streams	Kafka
Source	The database itself	Your app publishes explicitly
Retention	Oplog window (hours)	Days or weeks, configurable
Consumers	Each manages its own token	Consumer groups, partitions
Replay	Limited	Full replay from any offset

Change streams are simpler; Kafka is more durable. Use change streams for reacting to database changes within one system. Use Kafka when multiple services need the events, or when you need long retention and replay.

What the industry does

Change streams are used less than you would expect. Most teams already have Kafka or RabbitMQ and publish events explicitly from application code, because that gives cleaner control over the event contract than deriving it from raw database documents.

Where they genuinely shine:

- **CDC into a search index** — Elasticsearch or Algolia staying in sync with MongoDB. The most common real use.
- **Cache invalidation** — a document changes, evict the Redis key.
- **Legacy migration** — dual-write during a Strangler Fig migration. **Directly relevant to WeKan's business.**

The operational gotcha worth naming. A single-process change stream is a **single point of failure**. In a Kubernetes deployment with 5 pods, **all 5 open the stream and you process every change 5 times**.

Teams solve this with a leader-election lock in Redis, or by running the listener as a single-replica deployment. **Mentioning this is a strong signal** — it is the mistake people actually make.

SAY THIS

"Change streams let the application subscribe to database changes by tailing the oplog, so it requires a replica set. I would filter server-side with a \$match stage and persist the resume token so I can restart without missing events — same concept as a Kafka offset. The operational catch is that multiple pods each open their own stream, so it needs leader election or a single-replica deployment to avoid duplicate processing."

12. MongoDB in practice

- **Managed, not self-hosted.** Almost everyone runs **MongoDB Atlas** rather than managing replica sets themselves — failover, backups, monitoring and scaling are handled. Self-hosted MongoDB is mostly legacy or heavily regulated environments. **WeKan being a MongoDB partner means Atlas is what you would be working with.**
- **Sharding is rarer than people assume.** A single replica set handles a few TB comfortably. Most teams never shard. Reach for it only when data genuinely exceeds one machine, and expect it to be a project, not a config change.
- **Mongoose is near-universal in Node.** Schema definitions, validation, middleware hooks. The native driver is used for performance-critical paths, bulk operations, aggregations and migration scripts. If asked what you would use — say Mongoose with that caveat.
- **Indexes get created in migration scripts**, not by hand in a console. And built with `background: true` on existing collections, so you do not lock production.
- **Transactions are rare** — a handful per codebase, for payments and inventory.
- **Change streams are niche** — mostly CDC into a search index or cache invalidation.

Worth raising unprompted. WeKan recently partnered with **PowerSync** and **Ditto** for real-time data synchronisation between mobile and cloud, positioned as the alternative now that Atlas Device Sync and Realm are end-of-life. **Your AwanTunai CouchDB/PouchDB offline-first work maps directly onto that problem space** — offline-capable apps for merchants with unreliable connectivity. Bring it up.

Appendix — One-liners and self test

Every one-liner

Topic	The sentence
Sort without index	A sort without a supporting index becomes a blocking in-memory sort, capped at 32 MB. With the index it is an IXSCAN.
Index direction	Direction does not matter for a single field, since MongoDB can scan either way. For a compound index the sort must match the index direction exactly or be its complete inverse.
ESR	Equality first because it narrows to one contiguous block. Sort next so the index delivers documents already ordered. Range last, because a range spans many index values and destroys ordering for anything after it.
explain()	I check that totalDocsExamined is close to nReturned and that there is no SORT stage. A large gap means the index is not being used properly.
Leftmost prefix	One well-ordered compound index often replaces three separate ones — {city, amount, createdAt} also serves {city} and {city, amount}.
\$indexStats	Find indexes with zero ops, checking the since field so I am not misreading a recent restart. Unused indexes cost write throughput and RAM without returning anything.
WiredTiger cache	Indexes are on disk but must be paged into the cache. The working set — hot indexes plus hot documents — should fit in it.
Data modeling	Schema design is driven by access patterns, not normalization. Default to embed; reference when the child is queried independently, when the array is unbounded, or when the data is shared.
Bounded arrays	I would cap the embedded array with \$slice so the document size stays constant regardless of how much the child data grows.
Denormalization	I am trading write complexity for read performance, which is the right trade when reads outnumber writes by 1000 to 1.
Patterns	Extended reference and computed fields to keep reads to one query, bucketing for time-series, and the attribute pattern for highly variable fields.
Replication	Three nodes across three AZs. An odd number is needed so a majority can elect a new primary on failover.
Two nodes	With two nodes there is no majority after a failure, so the survivor cannot elect itself and the set goes read-only.
Read preference	Secondary reads for reporting only — replication lag breaks read-after-write consistency.
Repl vs shard	Replication is copies; sharding is splits. Replication solves availability, sharding solves data size.
Transactions	Single-document writes are already atomic, so my first move is schema design. Most SQL-style transaction needs disappear if the atomic unit is one document.
Concurrency	Move the condition into the query filter so the check and the update are one atomic operation, and treat a zero modifiedCount as the conflict signal.
Change streams	Tail the oplog, filter server-side with \$match, persist the resume token — same concept as a Kafka offset. Multiple pods each open their own stream, so it needs leader election.

Rapid-fire self test

Question	Where
What is an index, in one sentence, without jargon?	1
What happens when you sort on an unindexed field over 5 lakh documents?	1
Index is <code>{ amount: -1 }</code> . Does <code>sort({ amount: 1 })</code> use it?	1
Index is <code>{ city: 1, amount: -1 }</code> . Which of the three sort combinations fails?	1
Why does a range field destroy sorting for anything after it?	1
Which four <code>explain()</code> fields matter, and what is a good value for each?	2
What is a covered query and how do you spot it in explain output?	2
Index on <code>{ city, amount, createdAt }</code> — which sub-queries does it serve?	3
Two traps when reading <code>\$indexStats</code> output.	4
Are indexes stored in RAM or on disk? What is the WiredTiger cache default?	5
Why is an unused index worse than merely useless?	5
Three situations where embedding breaks.	6
Why should an order store the product price rather than reference it?	6
How do you keep an embedded array at exactly 3 elements?	6
Name the four schema patterns and what each solves.	7
What is the oplog, and why is it idempotent?	8
Primary dies in a 2-node set. What happens and why?	8
Difference between <code>w: 1</code> and <code>w: "majority"</code> ?	8
Replication vs sharding — one sentence each.	8
When do you genuinely need a transaction, and what are its three constraints?	9
Three concurrent <code>\$inc</code> on the same document — do any fail?	10
Show the code that causes a lost update, and the fix.	10
What does <code>__v</code> actually guard in Mongoose?	10
Why do change streams need a replica set?	11
Five Kubernetes pods running a change stream listener — what goes wrong?	11

Still to cover: aggregation pipelines — `$group`, `$project`, `$match` placement, `$lookup`, `$unwind`, `$facet`, and materialized views via `$out` and `$merge`. This is the remaining MongoDB gap for an SDE-3 round.